

Sample API Documentation - Rewrite an Existing Page

Documentation Improvement Exercise — OpenObserve Audit

I worked on an existing page for OpenObserve as my portfolio sample. The copyright and other information belongs to OpenObserve. This API page describes the JSON log ingestion endpoint. It is the most-used API in the entire OpenObserve platform.

The exercise follows three steps: present the original page as-is, identify exactly what is missing and why it matters, then rewrite the page from scratch following API reference best practices as used by Stripe, GitHub, Datadog, and Twilio.

Page Rewritten	Ingest Logs via JSON — API Reference
Source URL	https://openobserve.ai/docs/api/ingestion/logs/json/
Captured	March 2026
Audience	Application developers integrating OpenObserve directly via HTTP API
Framework	Diátaxis — Reference documentation (completeness, accuracy, scannability)
Why this page	It is the first API page a developer reaches. It has no auth, no response docs, no code examples, and no error guidance. It is the most critical page to fix.

Original Page — As Published

Source URL: <https://openobserve.ai/docs/api/ingestion/logs/json/>

JSON

Ingest logs in batch via POST `/api/{org}/{stream}/_json` using standard JSON. Supports flattening, timestamps, and up to 200 fields per record.

[Example payload — verbatim from live page]

```
[ { "kubernetes.annotations.kubectl.kubernetes.io/default-container": "prometheus",
  "kubernetes.annotations.kubernetes.io/psp": "eks.privileged", "kubernetes.container_hash":
  "quay.io/prometheus/prometheus@sha256:474...", "kubernetes.container_image":
  "quay.io/prometheus/prometheus:v2.39.1", "kubernetes.container_name": "prometheus", ... [20+
  more fields of equal complexity] } ]
```

[No explanation of the fields. No annotation. No simpler alternative. This is the first and only code example.]

Below is an example log before and after being flattened.

⚠ For performance reasons, OpenObserve limits the depth at which the JSON structure gets flattened. Past that limit, the generated field will contain unparsed JSON as a string. The default depth is 3, but this limit can be configured via the `ZO_INGEST_FLATTEN_LEVEL` environment variable.

By default we add a field `_timestamp` for each record with the value of NOW in microseconds (unix epoch value).

This will upload multiple records in batch with standard json format.

What this page is missing: No authentication instructions. No request headers documented. No URL path parameters explained. No request body schema. No response format or fields documented. No HTTP status codes. No curl, Python, Go, or JavaScript code examples. No error handling guidance. No rate limit information. No link to related endpoints (`_multi`, `_bulk`). The only code example is an overwhelming Kubernetes JSON blob with 20+ opaque field names and zero annotation.

Re-worked API documentation - Ingest Logs via JSON

Use this endpoint to send one or more log records to OpenObserve in standard JSON array format. This is the simplest way to ingest logs programmatically — no additional tooling or agents required. Records are indexed immediately and available for search within seconds of a successful response.

POST

/api/{organization}/{stream}/_json

 On this page:

Quick Start • Authentication • Endpoint & Parameters • Request Body • Response Format • HTTP Status Codes • Code Examples (curl, Python, Go, JavaScript) • JSON Flattening • Timestamp Handling • Error Handling • Rate Limits • Related Endpoints

Quick Start

Send your first log record in under 60 seconds. Replace the values in angle brackets with your own credentials and organization name.

CURL — QUICK START

```
curl -X POST \
  https://api.openobserve.ai/api/{organization}/default/_json \
  -H 'Content-Type: application/json' \
  -u 'your@email.com:YourPassword' \
  -d '[
    {
      "level": "info",
      "message": "User login successful",
      "user_id": "u_8821",
      "service": "auth-service",
      "duration_ms": 142
    }
  ]'
```

RESPONSE

Expected response

```
{
  "code": 200,
  "status": [
    { "name": "default", "successful": 1, "failed": 0 }
  ]
}
```

 **Self-hosted users:** Replace https://api.openobserve.ai with your own host, e.g. http://localhost:5080

Authentication

All API requests require HTTP Basic Authentication. Use your OpenObserve account email and password.

Method	Header	Value
Basic Auth	Authorization	Basic base64(email:password)
Content Type	Content-Type	application/json

 **Finding your credentials:** In OpenObserve Cloud, go to Data Sources in the left sidebar. Your ingestion credentials and org name are shown there. Copy the pre-filled curl command to get started immediately.

Endpoint & Parameters

Path Parameters

Parameter	Type	Required	Description
<code>{organization}</code>	string	Yes	Your organization name. Found in your OpenObserve account settings. Example: my-company
<code>{stream}</code>	string	Yes	The target log stream name. Stream is created automatically if it does not exist. Use lowercase letters, numbers, and hyphens. Example: production-logs

Query Parameters

This endpoint does not accept query parameters.

Request Headers

Parameter	Type	Required	Description
<code>Content-Type</code>	string	Yes	Must be application/json
<code>Authorization</code>	string	Yes	Basic authentication credentials as base64 (email:password)

Request Body

The request body is a JSON array containing one or more log record objects. Each object is an unstructured key-value map representing a single log event.

Body Schema

Parameter	Type	Required	Description
<code>[]</code> (root array)	array	Yes	JSON array of log record objects. Minimum 1 record. Maximum batch size is not published — keep batches under 5 MB for reliable performance.
<code>[].{field}</code>	any	No	Any key-value pair. Field names can be strings; values can be strings, numbers, booleans, or nested objects. Nested objects are flattened automatically (see JSON Flattening section).
<code>[]._timestamp</code>	integer	No	Unix timestamp in microseconds. If omitted, OpenObserve sets this to the server's current time. Provide this field to preserve original event time.

 **Field limit:** Each log record supports up to 200 fields after flattening. Flattening depth defaults to 3 levels. Both limits are configurable — see JSON Flattening below.

Minimal Example

```
JSON
[
  {
    "level": "error",
```

```
{
  "message": "Database connection timeout",
  "service": "user-service",
  "host": "prod-api-01"
}
```

Batch Example (multiple records)

JSON — BATCH WITH 3 RECORDS

```
[
  {
    "level": "info",
    "message": "Request received",
    "method": "GET",
    "path": "/api/users",
    "duration_ms": 23,
    "status_code": 200
  },
  {
    "level": "warn",
    "message": "Slow query detected",
    "query": "SELECT * FROM orders WHERE...",
    "duration_ms": 4821,
    "_timestamp": 1711324800000000
  },
  {
    "level": "error",
    "message": "Payment gateway timeout",
    "error_code": "GW_TIMEOUT",
    "transaction_id": "txn_9f2a1c"
  }
]
```

Response Format

On success, the API returns a JSON object summarising the ingestion result per stream. Check the failed field — a value greater than 0 means some records were rejected.

SUCCESS RESPONSE — HTTP 200

```
{
  "code": 200,
  "status": [
    {
      "name": "production-logs",
      "successful": 3,
      "failed": 0
    }
  ]
}
```

Response Fields

Parameter	Type	Required	Description
code	integer	—	HTTP status code mirrored in the response body.
status	array	—	One entry per target stream.
status[].name	string	—	The stream name the records were written to.
status[].successful	integer	—	Number of records successfully ingested.
status[].failed	integer	—	Number of records rejected. Always check this field — a partial failure returns HTTP 200 but with failed > 0.

HTTP Status Codes

Code	Status	Meaning
200	OK	All or some records ingested. Check status[].failed for partial failures.
400	Bad Request	Request body is malformed. Body is not a valid JSON array, or a field value is of an incompatible type.
401	Unauthorized	Missing or invalid authentication credentials.
403	Forbidden	Credentials are valid but the user does not have write access to this organization or stream.
413	Payload Too Large	Request body exceeds the maximum allowed size.
429	Too Many Requests	Rate limit exceeded. See Rate Limits section.
500	Internal Server Error	Unexpected server-side error. Retry with exponential backoff.

Code Examples for different languages

curl

```
CURL
curl -X POST \
  'https://api.openobserve.ai/api/my-org/app-logs/_json' \
  -H 'Content-Type: application/json' \
  -u 'admin@mycompany.com:MySecurePassword' \
  -d '[
    {
      "level": "info",
```

```
    "message": "Order placed",
    "order_id": "ord_1234",
    "amount": 89.99,
    "currency": "USD"
  }
]
```

Python

PYTHON (REQUESTS)

```
import requests
import json

ENDPOINT = 'https://api.openobserve.ai/api/my-org/app-logs/_json'
CREDENTIALS = ('admin@mycompany.com', 'MySecurePassword')

logs = [
    {
        "level": "info",
        "message": "Order placed",
        "order_id": "ord_1234",
        "amount": 89.99,
        "currency": "USD"
    },
    {
        "level": "error",
        "message": "Payment declined",
        "order_id": "ord_1235",
        "error_code": "CARD_DECLINED"
    },
]

response = requests.post(
    ENDPOINT,
    auth=CREDENTIALS,
    headers={'Content-Type': 'application/json'},
    data=json.dumps(logs)
)

result = response.json()
if result['status'][0]['failed'] > 0:
    print(f"Warning: {result['status'][0]['failed']} records failed")
else:
    print(f"Ingested {result['status'][0]['successful']} records successfully")
```

Go

```
GO

package main

import (
    "bytes"
    "encoding/json"
    "fmt"
    "net/http"
)

func main() {
    endpoint := "https://api.openobserve.ai/api/my-org/app-logs/_json"
```

```
logs := []map[string]interface{}{
    {
        "level":    "info",
        "message":  "Order placed",
        "order_id": "ord_1234",
        "amount":   89.99,
    },
}

body, _ := json.Marshal(logs)

req, _ := http.NewRequest("POST", endpoint, bytes.NewBuffer(body))
req.SetBasicAuth("admin@mycompany.com", "MySecurePassword")
req.Header.Set("Content-Type", "application/json")

resp, err := http.DefaultClient.Do(req)
if err != nil {
    fmt.Printf("Request failed: %v\n", err)
    return
}
defer resp.Body.Close()

var result map[string]interface{}
json.NewDecoder(resp.Body).Decode(&result)
fmt.Printf("Response: %+v\n", result)
}
```

JavaScript (Node.js / fetch)

```
JAVASCRIPT / NODE.JS (FETCH)
const logs = [
    {
        level: "info",
        message: "Order placed",
        order_id: "ord_1234",
        amount: 89.99,
    },
];

const credentials = btoa('admin@mycompany.com:MySecurePassword');

const response = await fetch(
    'https://api.openobserve.ai/api/my-org/app-logs/_json',
    {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'Authorization': `Basic ${credentials}`,
        },
        body: JSON.stringify(logs),
    }
);

const result = await response.json();
console.log(`Ingested: ${result.status[0].successful}, Failed: ${result.status[0].failed}`);
```

JSON Flattening

OpenObserve automatically flattens nested JSON objects into dot-separated field names before indexing. This makes nested data queryable with standard SQL and search without any manual schema configuration.

Before and After Flattening

JSON — INPUT

```
// INPUT — nested JSON
{
  "actor": {
    "ip": "203.0.113.42",
    "id": 558875,
    "parent": {
      "id": 45516,
      "active": true
    }
  },
  "response": {
    "error_occurred": false,
    "status_code": 200
  }
}
```

JSON — STORED & INDEXED

```
// OUTPUT — flattened (what gets stored and indexed)
{
  "actor_ip": "203.0.113.42",
  "actor_id": 558875,
  "actor_parent_id": 45516,
  "actor_parent_active": true,
  "response_error_occurred": false,
  "response_status_code": 200
}
```

Configuring Flatten Depth

By default, OpenObserve flattens up to 3 levels deep. Objects nested beyond this depth are stored as a raw JSON string in a single field.

Environment Variable	Behaviour
<code>ZO_INGEST_FLATTEN_LEVEL=3</code>	Default. Flattens up to 3 levels. Objects deeper than 3 levels are stored as a JSON string.
<code>ZO_INGEST_FLATTEN_LEVEL=0</code>	Disables the depth limit. All nesting is flattened regardless of depth. Use with caution on deeply nested structures.
<code>ZO_INGEST_FLATTEN_LEVEL=N</code>	Any positive integer N. Flattens up to N levels deep.

Timestamp Handling

OpenObserve uses the `_timestamp` field to order and query records by time. Understanding how this works is important for correct time-range queries.

Scenario	<code>_timestamp</code> value	Behaviour
Not provided	Server time (NOW)	Timestamp is set to server ingestion time in microseconds (Unix epoch). Use this for real-time log shipping.

Scenario	_timestamp value	Behaviour
Provided	Your value (microseconds)	Use this for backfill, batch re-ingestion, or when your logs already have original event timestamps. Must be Unix epoch in microseconds.

JSON — CUSTOM TIMESTAMP

```
// Setting a custom timestamp (Unix epoch microseconds)
{
  "level": "error",
  "message": "Backfilled historical event",
  "_timestamp": 1711324800000000
  // ↑ Equals: 2024-03-25 00:00:00 UTC
  // Formula: Date.getTime() * 1000 (milliseconds → microseconds)
}
```

Error Handling

Always check both the HTTP status code and the `status[].failed` field in the response body. A 200 response with `failed > 0` indicates a partial failure — some records were accepted, others were rejected.

Common Errors and Fixes

Code	Likely Cause	Fix
400	Body is not a JSON array, or contains invalid JSON	Ensure the body is a JSON array [...], not a single object {...}. Validate JSON before sending.
401	Wrong credentials or missing Authorization header	Check email/password. Ensure the Authorization header is correctly base64-encoded as email:password.
413	Request body too large	Split the batch into smaller requests. Target under 5 MB per request for best reliability.
429	Rate limit exceeded	Implement exponential backoff. Batch records to reduce request frequency. Check your plan limits.

Rate Limits

Rate limits depend on your OpenObserve plan. When a rate limit is exceeded, the API returns HTTP 429. Implement exponential backoff with jitter in your ingestion clients.

- **Cloud plans:** Refer to your plan details at openobserve.ai/pricing for ingestion rate limits.
- **Self-hosted:** No built-in rate limit by default. Configure limits via your reverse proxy (Nginx, Traefik) or load balancer.
- **Recommended batch size:** 100–1,000 records per request, with requests no more frequent than every 1 second per client.

Related Endpoints

Method	Endpoint	Use When
POST	/api/{org}/{stream}/_json	This page. Standard JSON array. Simplest integration.
POST	/api/{org}/{stream}/_multi	JSON Lines (NDJSON) format — one JSON object per line. More memory-efficient for very large batches.
POST	/api/{org}/_bulk	Elasticsearch-compatible bulk API. Use when migrating from Elasticsearch or using ES client libraries.

Change Log — What Changed and Why

Source URL: <https://openobserve.ai/docs/api/ingestion/logs/json/>

#	Change	Original State	Rationale
1	Added Authentication section	No authentication documentation preset. Users had to discover credentials from the Getting Started guide in a separate section.	Authentication is the #1 point of failure for API first-timers. Every API reference page must include it. Auth docs must live on the same page as the endpoint — not require navigation elsewhere.
2	Added endpoint badge + path parameter table	Method (POST) and URL template were buried in text. Path parameters {org} and {stream} were never explained.	Visual endpoint badge (POST + URL) is now the industry standard (Stripe, GitHub, Twilio). Path parameter tables eliminate the most common 404 errors from users guessing what {org} means.
3	Replaced Kubernetes example with simple, clear examples	The only example was a raw Kubernetes log with 20+ long field names like <code>kubernetes.annotations.kubectl.kubernetes.io/default-container</code> . This obscures the API structure behind implementation detail.	API examples should teach the concept, not a specific production scenario. Short, readable examples with 4–5 fields communicate the structure instantly. Production complexity belongs in integration guides.
4	Added multi-language code examples	No code examples for any language. No curl example on the API reference page itself (it existed elsewhere but was not linked).	curl, Python, Go, and JavaScript cover the majority of OpenObserve's developer audience. Copy-paste examples are the single highest-value content type in an API reference.
5	Added response format + response field documentation	No response documentation. Users did not know what a successful response looked like, what fields it contained, or how to detect partial failures.	The <code>status[].failed > 0</code> partial failure case is particularly important — HTTP 200 does not mean all records were ingested. Without response documentation, users never know to check this field.
6	Added HTTP status codes table	No status code documentation at all. Users encountering 401, 403, or 413 had no in-page guidance.	Status code tables with fix guidance are the most-referenced section of any API page after the code examples. They deflect a large portion of API-related support requests.
7	Added on-page navigation (contents list)	Original page had no internal navigation. Users could not jump to the section they needed.	API reference pages are scanned, not read linearly. A contents list at the top allows users to jump directly to Authentication, Code Examples, or Error Handling without scrolling past irrelevant sections.
8	Added Related Endpoints section	No cross-linking to <code>_multi</code> or <code>_bulk</code> APIs. Users were unaware that alternatives existed for their specific use case.	The <code>_bulk</code> endpoint is critical for Elasticsearch migrators, and <code>_multi</code> is better for large NDJSON workloads. Without cross-links, users use <code>_json</code> for everything, including cases where a better option exists.

 **Overall Rationale:** The original page communicated what the endpoint does but failed to communicate how to use it. Every section added — authentication, path parameters, request schema, response format, status codes, multi-language examples, error handling — is standard in every well-regarded API reference (Stripe, Twilio, GitHub, Datadog). The rewrite follows the Diátaxis framework: this is a Reference page, so it prioritizes completeness and accuracy over narrative prose.

API Page Rewrite — OpenObserve • Vivek Deshmukh • March 2026